

Inkrementerings- och dekrementeringsoperatorerna ++ och □

C++ baseras sig som bekant på C, och C har alltid varit alla lata programmerares favoritspråk. Det finns en mängd förkortningar och olika sätt att skriva saker i C, och då även i C++. En operation som man märkte att ofta förekom i tidiga C-program var en inkrementering eller dekrementering av ett tal med 1, t.ex:

```
int Index, Ar;
Index = Index - 1;
Ar = Ar + 1;
```

För att göra det hela kortare att skriva uppfanns operatorerna ++ och >□. Dessa används för att inkrementera eller dekrementera sitt argument. De ovanstående exemplen skulle med dessa operatorer kunna bli:

```
int Index, Ar;
Index>□;
Ar++;
```

En hel del kortare eller hur? För att göra det mera intressant kan man placera operatorerna antingen *framför* eller *bakom* operanden. Om operatorerna är framför kallas dem för *prefixa* och är de bakom är de *postfixa*. Beroende på placeringen påverkas *när* inkrementeringen/dekrementeringen utförs. En prefixoperator evalueras *före* operanden används i andra uttryck, medan en postfixoperator evalueras *efter* att operanden använts i andra uttryck. Således är dessa två uttryck inte ekvivalenta:

```
int Tal1 = 10;
int Tal2 = 10;
cout << "Tal1++ = " << Tal1++ << endl; // ger svaret 10
cout << "++Tal2 = " << ++Tal2 << endl; // ger svaret 11
```

Det första fallet skriver ut `Tal1` innan det inkrementeras och ger svaret 10, medan det senare exemplet inkrementerar före `Tal2` används, och ger således svaret 11. Man bör vara medveten om denna fundamentala skillnad då man använder dessa operatorer. De är dock mycket praktiska att ha då man t.ex. itererar (se [Kapitel 5](#)) och vill öka/minska en indexvariabel med 1.